

DOCUMENTATION TECHNIQUE

Documentation Technique & Architecture du Pipeline

Table des Matières

- CLI To-Do List Manager - Technical Specification & Architecture Document 3**
 - 1. Executive Summary & Architecture Overview 3
 - 1.1 Executive Brief 3
 - 1.2 Maturity Assessment 3
 - 1.3 Technical Stack 3
 - 1.4 Architectural Constraints 3
 - 1.5 Critical Dependencies 3
 - 2. Architecture Workflows 4
 - 2.1 Project Implementation Roadmap & Traceability 6
 - 2.2 CLI Command Execution Workflow 6
 - 2.3 CLI To-Do Manager Data Model 7
 - 2.4 User Story Interaction Sequence 8
 - 3. Detailed Technical Specifications & Business Rules 9
 - 3.1 Requirements Traceability 9
 - 3.2 Security Rules 12
 - 3.3 Data Models 12
 - 4. Project Governance & Structural Gaps 12
 - 4.1 Structural Gaps 12
 - 4.2 Remediation & Workflow 13
 - 5. Technical & Domain Glossary (Terminology Reference) 13

CLI To-Do List Manager - Technical Specification & Architecture Document

1. Executive Summary & Architecture Overview

1.1 Executive Brief

The CLI To-Do List Manager is a Python-based command-line utility designed for task lifecycle management. It utilizes a local JSON file (`~/ . todos . json`) for persistent storage, implementing a service-layer architecture to decouple CLI dispatching from data serialization and storage I/O. The system focuses on three core user stories: task creation/completion, filtered listing (human-readable and JSON), and targeted or bulk removal of tasks.

1.2 Maturity Assessment

The project is READY for execution. The structural integrity is high, with a clear phase-based roadmap and explicit task dependencies. While there are minor gaps regarding formal acceptance criteria and OS-level file permission constraints, the presence of 'Independent Tests' for each user story provides sufficient validation gates for the current scope.

1.3 Technical Stack

- Python
- pyproject.toml

1.4 Architectural Constraints

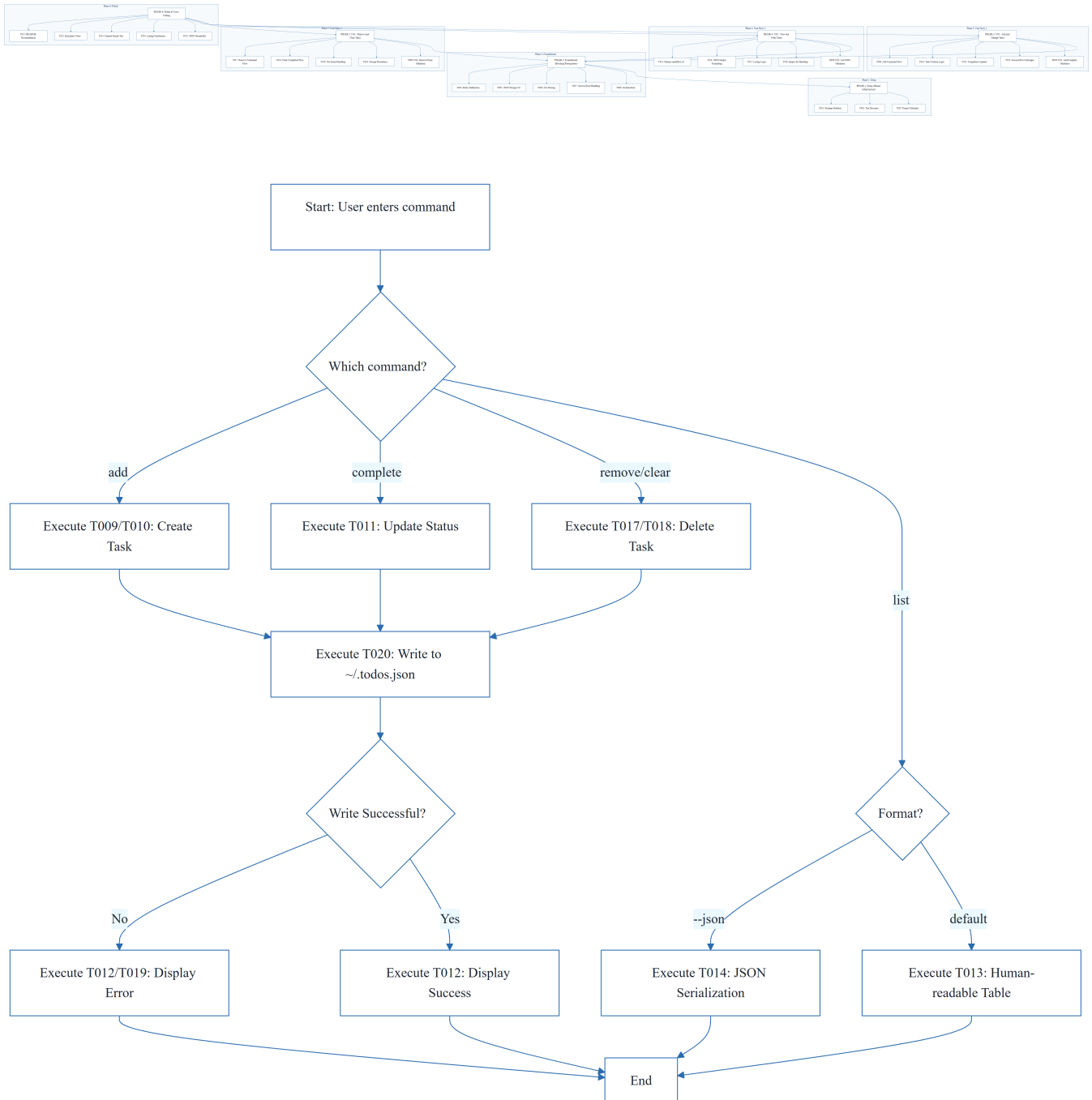
- Sequential ID assignment for task creation.
- Strict JSON serialization for data persistence.
- Human-readable and JSON output modes for listing.
- Storage path resolution targeting `~/ . todos . json`.
- Blocking dependency: Foundational phase must be complete before any User Story implementation.
- Manual smoke test validation for all core commands (add, list, complete, remove, clear).

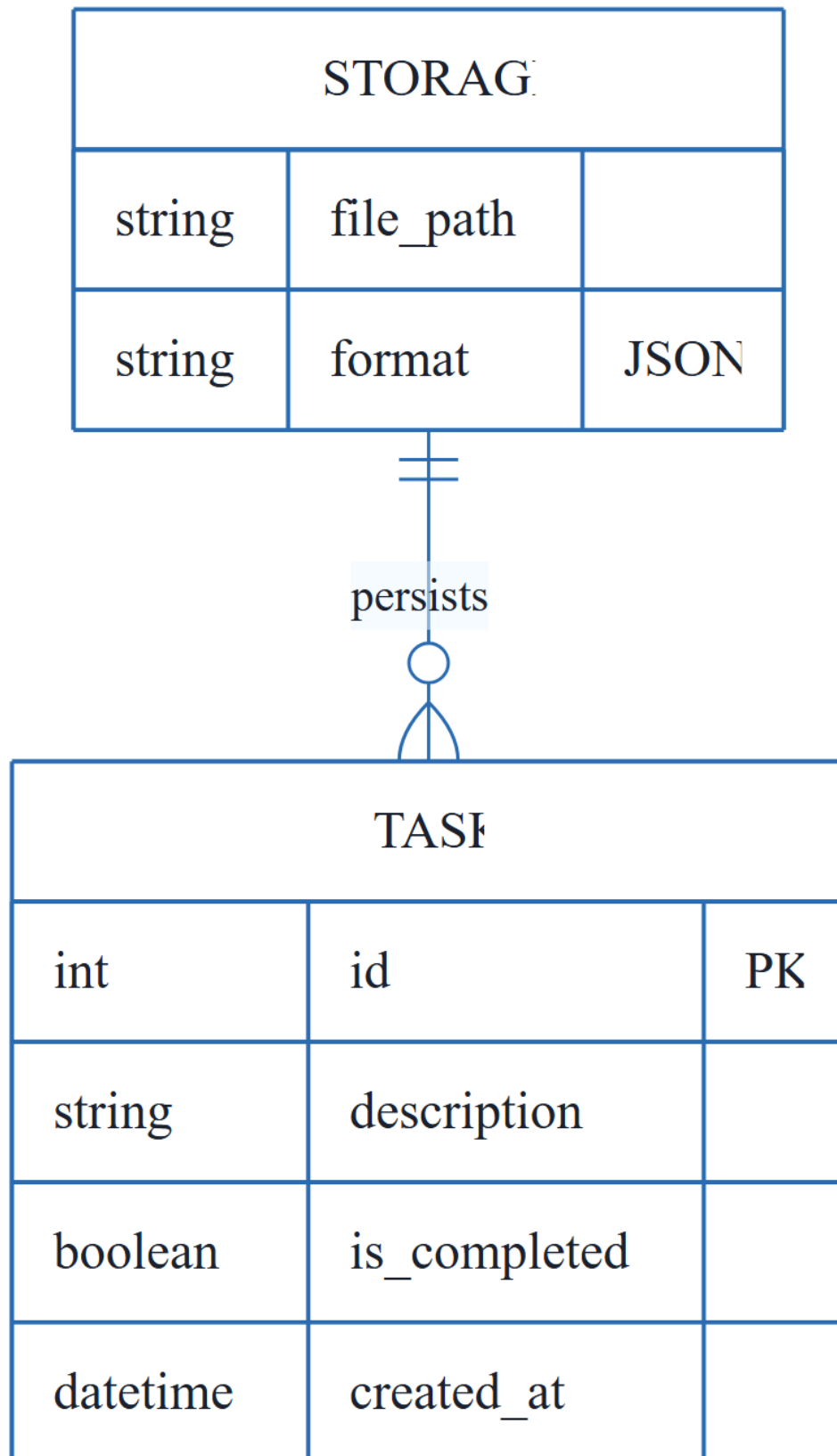
1.5 Critical Dependencies

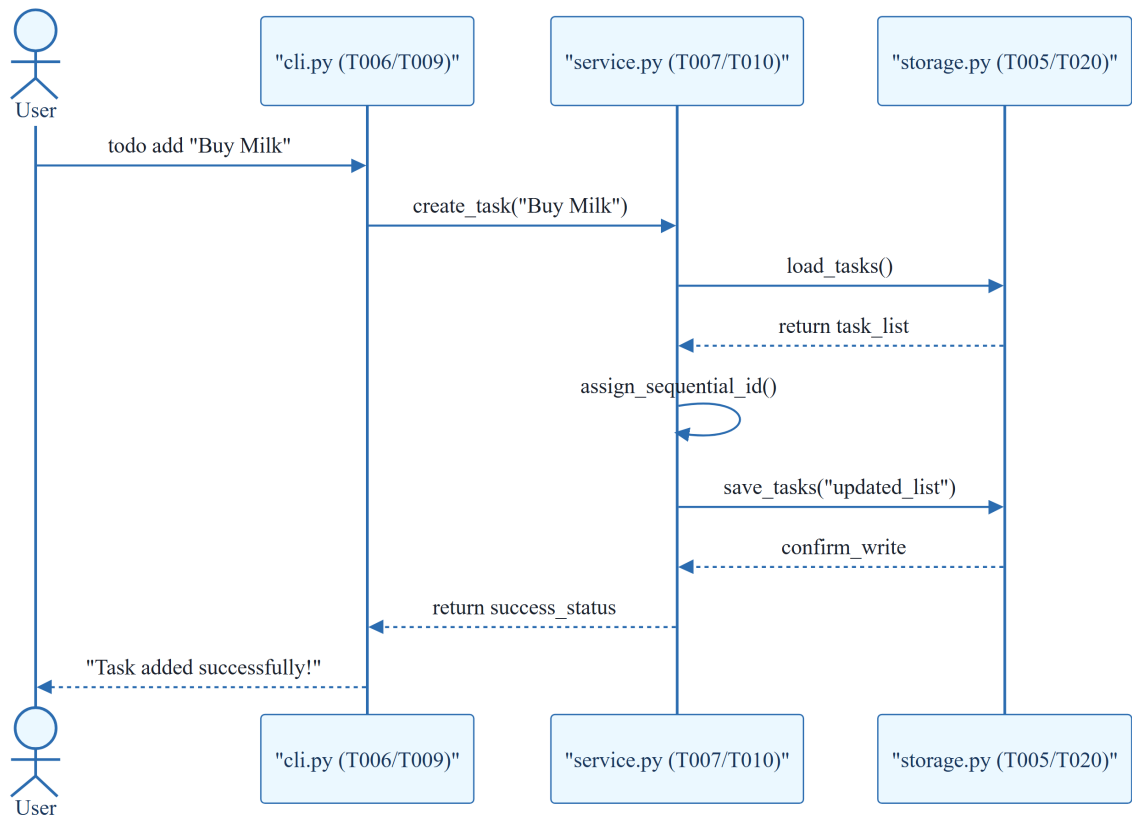
- JSON file system I/O for `~/ . todos . json`.
- Python package skeleton (`src/todo_manager/`).
- Sequential dependency: Setup $\rightarrow$ Foundational $\rightarrow$ User Stories $\rightarrow$ Polish.

- Internal data integrity: Task entity serialization helpers in `models.py` must precede service-layer implementation.
- CLI dispatch logic in `cli.py` as the primary entry point for all user stories.

2. Architecture Workflows

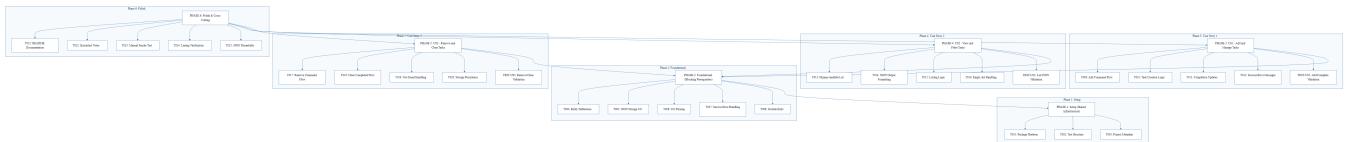




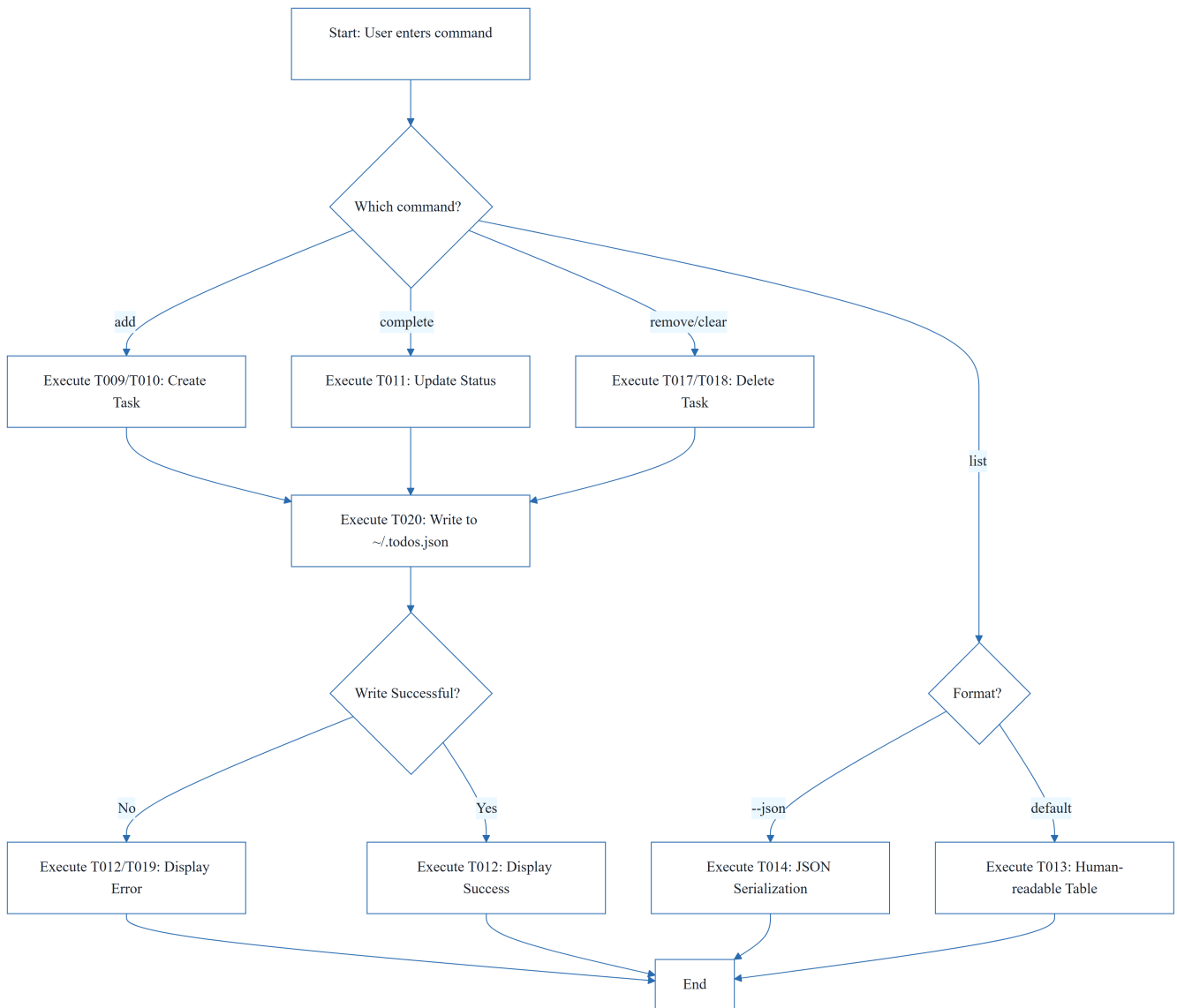


& Visual Diagrams

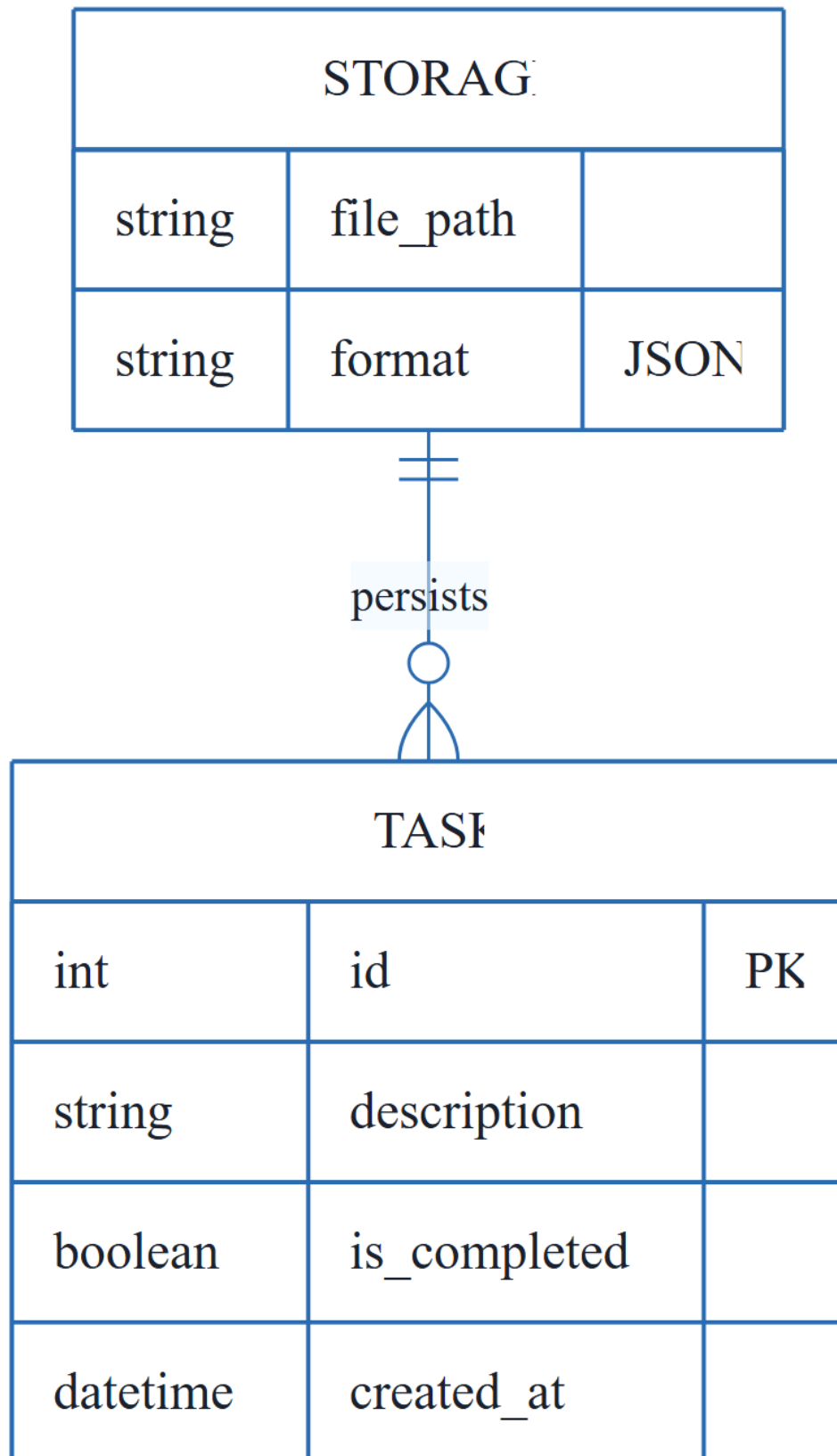
2.1 Project Implementation Roadmap & Traceability



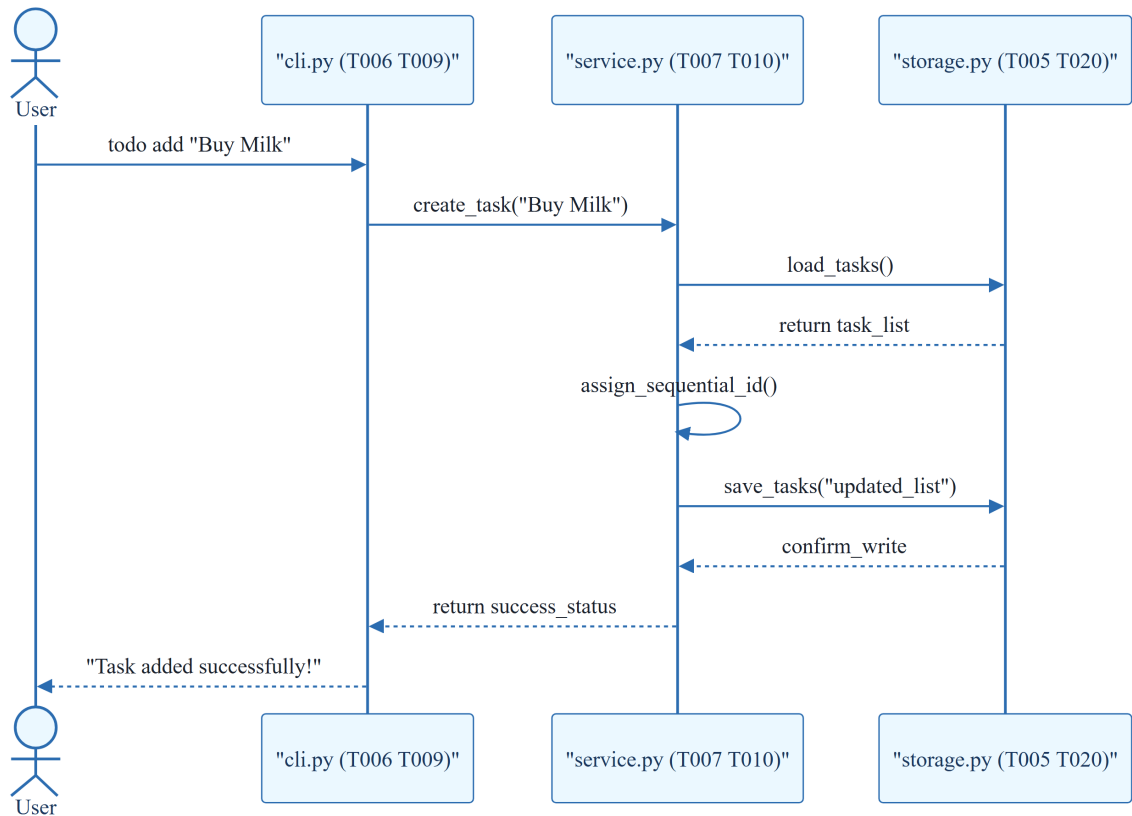
2.2 CLI Command Execution Workflow



2.3 CLI To-Do Manager Data Model



2.4 User Story Interaction Sequence



3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

ID	Description	Source Phase	Status
T001	Create the Python package skeleton in <code>src/todo_manager/init.py</code> , <code>main.py</code> , <code>cli.py</code> , <code>models.py</code> , <code>storage.py</code> , and <code>service.py</code>	PHASE-1	completed
T002	Create the test directory structure in <code>tests/unit/</code> and <code>tests/integration/</code>	PHASE-1	completed
T003	Add project metadata and tooling entry points in <code>pyproject.toml</code>	PHASE-1	completed
T004	Implement task entity definitions and serialization helpers in <code>src/todo_manager/models.py</code>	PHASE-2	completed

ID	Description	Source Phase	Status
T005	Implement JSON storage path resolution and file I/O helpers in <code>src/todo_manager/storage.py</code>	PHASE-2	completed
T006	Implement shared command-line parsing and top-level dispatch in <code>src/todo_manager/cli.py</code>	PHASE-2	completed
T007	Implement shared service-layer error handling and task collection utilities in <code>src/todo_manager/service.py</code>	PHASE-2	completed
T008	Define module entry behavior for python <code>-m todomanager</code> in <code>src/todomanager/main.py</code>	PHASE-2	completed
T009	Implement the add command flow in <code>src/todomanager/cli.py</code> and <code>src/todomanager/service.py</code>	PHASE-3	pending
T010	Implement task creation, sequential ID assignment, and <code>created_at</code> population in <code>src/todo_manager/models.py</code>	PHASE-3	pending
T011	Implement completion updates for existing tasks in <code>src/todo_manager/service.py</code>	PHASE-3	pending
T012	Add user-facing success and error messages for add and complete operations in <code>src/todo_manager/cli.py</code>	PHASE-3	completed
T013	Implement human-readable task listing output in <code>src/todo_manager/cli.py</code>	PHASE-4	pending
T014	Implement <code>--json</code> output formatting in <code>src/todomanager/cli.py</code> using the task serialization helpers in <code>src/todomanager/models.py</code>	PHASE-4	pending
T015	Add listing logic that preserves task order and status fields in <code>src/todo_manager/service.py</code>	PHASE-4	pending

ID	Description	Source Phase	Status
T016	Handle the empty-list case with a clear message in <code>src/todo_manager/cli.py</code>	PHASE-4	pending
T017	Implement the <code>remove</code> command flow in <code>src/todomanager/cli.py</code> and <code>src/todomanager/service.py</code>	PHASE-5	pending
T018	Implement the <code>clear</code> command flow for removing completed tasks in <code>src/todo_manager/service.py</code>	PHASE-5	pending
T019	Add not-found handling for invalid task IDs in <code>src/todo_manager/cli.py</code>	PHASE-5	pending
T020	Ensure storage writes persist removals and clears safely in <code>src/todo_manager/storage.py</code>	PHASE-5	pending
T021	Document the CLI usage and storage behavior in <code>README.md</code>	PHASE-6	pending
T022	Add quickstart verification notes and examples in <code>specs/001-cli-todo-manager/quickstart.md</code>	PHASE-6	pending
T023	Run a manual smoke test of <code>add</code> , <code>list</code> , <code>complete</code> , <code>remove</code> , and <code>clear</code> against <code>~/todos.json</code>	PHASE-6	pending
T024	Verify linting and formatting expectations for the source files in <code>src/todo_manager/</code>	PHASE-6	pending
T025	Confirm the generated JSON output remains parseable for the todo list <code>--json</code> path	PHASE-6	pending
TEST-US1	Run <code>todo add "Task description"</code> and <code>todo complete</code> , then confirm the stored task list reflects the new task and its completion status.	PHASE-3	pending

ID	Description	Source Phase	Status
TEST-US2	Run <code>todo list</code> and <code>todo list --json</code> and confirm the output is readable or parseable JSON while showing the full task collection.	PHASE-4	pending
TEST-US3	Run <code>todo remove</code> and <code>todo clear</code> , then confirm the targeted task or completed tasks are removed while other tasks remain intact.	PHASE-5	pending

3.2 Security Rules

- **File Permissions:** The application must ensure that `~/ .todos.json` is accessible only by the current user to prevent unauthorized task modification.
- **Input Validation:** CLI inputs for task descriptions must be sanitized to prevent injection or corruption of the JSON storage file.

3.3 Data Models

The system utilizes a flat JSON structure stored in the user's home directory.

- **Task Entity:**
 - `id` (Integer): Primary Key, sequentially assigned.
 - `description` (String): The text of the task.
 - `is_completed` (Boolean): Completion status.
 - `created_at` (DateTime): ISO 8601 timestamp of creation.
- **Storage Entity:**
 - `file_path` (String): Absolute path to `~/ .todos.json`.
 - `format` (String): Fixed as "JSON".

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Gap	Priority	Remediation Advice
Acceptance Criteria	MEDIUM	While 'Independent Tests' are provided, formal acceptance criteria for each user story should be explicitly listed.

Gap	Priority	Remediation Advice
Security & Performance Constraints	LOW	Define constraints regarding JSON file size or OS-level file permissions for <code>~/todos.json</code> .
Open Questions & Uncertainties	LOW	No open questions were listed in the source; verify if any edge cases (e.g., corrupted JSON) need addressing.

4.2 Remediation & Workflow

1. **Validation Phase:** Integrate formal Acceptance Criteria into the TEST-USx identifiers.
2. **Hardening Phase:** Implement file permission checks during the T005 (Storage I/O) implementation.
3. **Edge Case Analysis:** Add a task to PHASE-6 to specifically test corrupted JSON recovery.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
Checkpoint	TECHNICAL_STACK	PHASE-2	A synchronization gate ensuring all blocking infrastructure is verified before parallel feature development commences.
Foundational	TECHNICAL_STACK	PHASE-2	The shared infrastructure layer containing core entity definitions and I/O helpers that block all subsequent user story implementation.
Goal	BUSINESS_DOMAIN	PHASE-3	The high-level functional objective a user must achieve within a specific feature set.
ID	BUSINESS_DOMAIN	T010	A sequential alphanumeric token assigned to each entry to enable unique targeting for updates or removals.
JSON	TECHNICAL_STACK	T005	The lightweight data-interchange format used for persistent storage in the home directory and for scriptable output.
MVP	BUSINESS_DOMAIN	PHASE-3	The minimum viable product consisting of the ability to create and mark entries as finished.

Term	Category	Context Anchor	Project Definition
Organization	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The structural grouping of implementation units by user story to ensure independent testability.
Prerequisites	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The set of mandatory design documents including data models and command contracts required before coding.
README	TECHNICAL_STACK	T021	The primary documentation file detailing usage instructions and storage behavior.
Setup	TECHNICAL_STACK	PHASE-1	The initial phase involving package skeleton creation and tooling configuration in the project metadata file.
T016	TECHNICAL_STACK	T016	The specific implementation unit responsible for providing a clear message when the collection is empty.
Tests	TECHNICAL_STACK	T002	The verification suite divided into unit and integration directories to validate system behavior.
population in	TECHNICAL_STACK	T010	The process of assigning a timestamp to the creation field during entity instantiation.
todo clear	BUSINESS_DOMAIN	T018	The operational command that purges all entries marked as finished from the persistent store.
todo list	BUSINESS_DOMAIN	T013	The operational command that retrieves and displays all entries in either human-readable or machine-parseable format.
■■ CRITICAL	TECHNICAL_STACK	PHASE-2	A high-priority constraint indicating that no feature work can proceed until the current phase is fully validated.